GOVERNMENT COLLEGE OF ENGINEERING AURANGABAD (Chhatrapati Sambhajinagar)

(An Autonomous Institute of Govt. of Maharashtra) Station Road, Osmanpura, Aurangabad-431005 (M.S.)
(0240) 2366101, 2366111, Fax: (0240)2332835

Object Oriented Programming Lab Manual

Course Code - CSPCC2010
Department of Computer Science & Engineering
NEP
Second Year CSE (UG Program)

CSPCC2010:Lab Object Oriented Programming		
Teaching Scheme	**Examination Scheme**	
Practical: 2Hrs/Week	**ISE I (Term Work)**	**25 Marks**
Credits:01	**End Semester Evaluation**	**25 Marks**

(Effective from: A.Y. 2022-2023)

Course Outcomes: After completion of this course students will be able to:

Course Outcomes

CO1 Execute JAVA programsbasedonsimpleconstructslikearrays,loops,decision Statements, functions etc

CO2 Incorporateobjectorientedconceptslikeclasses,objects,inheritance, polymorphism resembling a real time situation.

CO3 Demonstratetheuseofpackagesandinterfaces.

CO4 DevelopOOPprograms containingUsercreated ExceptionhandlingThreading.

CO5 ConnectwithJavadevelopmentEnvironmentlichasEclipse,NetBeans, IntelliJetc.

List of the Experiments:

The student shall perform minimum ten experiments of the following using Java

Sr. No.	Title of the Experiments	Skill / Knowledge Level	CO
1	Write a Java program to convert time in seconds to hours, minutes and seconds and display the output in format HH:MM:SS	S1	CO1
2	Write a Java program to rearrange all the elements of a given array of integers so that all the odd numbers come before all the even numbers.	S2	CO2
3	Write a Java program that accepts three integers from the user and return true if two or more of them (integers) have the same rightmost digit. The integers are non- negative.	S2	CO2
4	Create a Java class called Complex with the following details as member variables within it. (i) Real (ii) Imaginary. Develop a Java program to	S2	CO2

	perform addition and subtraction of two complex numbers by using the method add() and subtract( ) respectively, by passing object as parameter and display result using method display(). Initialize the real and imaginary values of the complex number using parameterized constructor. Also demonstrate overloading constructors and methods.		
5	Design an Address class with member variables Street num, city, state and country and appropriate constructor. Design a Student class with constructor (Student (String USN, String Name, String addr)), College class with constructor (College (String Name, String addr)) and Employee class with constructor (Employee (String EmpID, String Name, Stringaddr)). Write a Java program to create 'n' Student objects, College Objects andEmployee objects and print the student, college and employee addresses respectively and demonstrate passing of object as a parameter to the constructor.	S2	CO2
6	**Abstract classes, Inheritance & polymorphism:** Write a java program to create an abstract class named shape that contains two integers and an empty method named printArea(). Create three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class shape. Each one of the class contains only the method printArea() that print the area of the given shape	S3	CO4
7	**Exception Handling:** Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw Number Format Exception. If Num2 were Zero, the program would throwan Arithmetic Exception. Display the exception in a message dialog box.	S3	CO3
8	**Multithreading:** Write a Java program that implements a multithreaded program with three threads. First thread generates a random integer every 1 second and if the value is even, second thread computes the square of the number and prints. If the value is odd the third thread will print the value of cube of the number.	S3	CO4
9	File Handling Problem statement	S3	CO4
10	Write a Java program for the following: i) Create a doubly linked list of elements. ii) Delete a given element from the above list. iii) Display the contents of the list after deletion	S3	CO4
11	Write a java program to simulate a traffic light. The program lets the user select one of the three lights: red, yellow or green. On selecting a button, an appropriate message with "Stop" or "Ready" or " Go" should appear	S2	CO4

	above the buttons selected color.		
12	Write a Java program that implements Quick sort algorithm for sorting a list of names in ascending order.	S3	CO4

Assessment:

ISE I (Term Work) : In-Semester Evaluation of 25 marks each will be based on practical assignments completed and timely submission

End Semester Evaluation:The ESE will be based on oral / practical performance of the students

Practical - 1

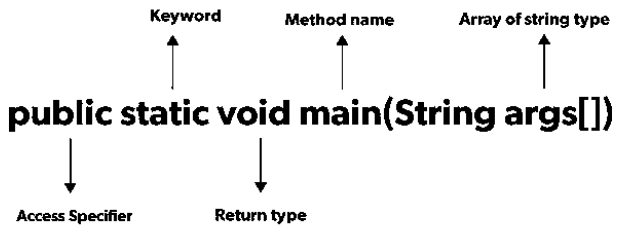
Aim :Write a Java program to convert time in seconds to hours, minutes and seconds and display the output in format HH:MM:SS

Theory :

In Java programs, the point from where the program starts its execution or simply the entry point of Java programs is the main() method. Hence, it is one of the most important methods of Java and having a proper understanding of it is very important.

The Java compiler or JVM looks for the main method when it starts executing a Java program. The signature of the main method needs to be in a specific way for the JVM to recognize that method as its entry point. If we change the signature of the method, the program compiles but does not execute.

The execution of the Java program, the java.exe is called. The Java.exe inturn makes Java Native Interface or JNI calls, and they load the JVM. The java.exe parses the command line, generates a new String array, and invokes the main() method. A daemon thread is attached to the main method, and this thread gets destroyed only when the Java program stops execution.



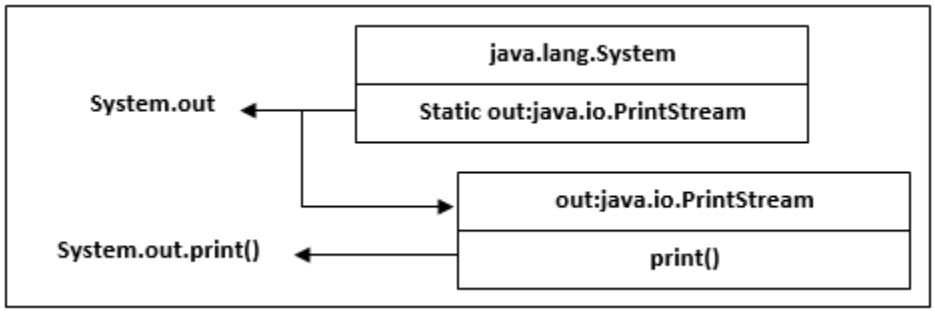
Java System.out.println() is used to print an argument that is passed to it. The statement can be broken into 3 parts which can be understood separately as:

1) System: It is a final class defined in the java.lang package.

2)out: This is an instance of PrintStream type, which is a public and static member field of the System class.

3)println(): As all instances of PrintStream class have a public method println(), hence we can invoke the same on out as well. This is an upgraded version of print(). It prints any argument passed to it and adds a new line to the output. We can assume that System.out represents the

Standard Output Stream.



Program :

```
import java.io.DataInputStream;
import java.io.IOException;
class Practical1
{
public static void main(String args[])throws IOException
{
int time;
DataInputStream dis=new DataInputStream(System.in);
System.out.print("Enter a Time:");
time=Integer.parseInt(dis.readLine());
System.out.println("Time is :"+new Time(time).getTime());
}
}
class Time
{
int hours;
int minutes;
int seconds;
int time;
public Time(int t)
{
time=t;
while(t>0)
{
 if(time/3600>0)
 {
 hours++;
```

```
 time=time-3600;
 }
 else if(time/60>0)
 {
 minutes++;
 time=time-60;
 }
 else
 {
 seconds=time;
 break;
 }
}
}
 public String getTime()
{
 String ans="";
 if(hours>9)
 {
 if(minutes>9)
 {
 if(seconds>9)
 {
ans=""+hours+":"+minutes+":"+seconds;
 }
 else
 {
ans=""+hours+":"+minutes+":0"+seconds;
 }
 }
 else
 {
 if(seconds>9)
 {
ans=""+hours+":0"+minutes+":"+seconds;
 }
 else
 {
ans=""+hours+":0"+minutes+":0"+seconds;
 }
```

```
 }
 }
 else
 {
 if(minutes>9)
 {
 if(seconds>9)
 {
ans="0"+hours+":"+minutes+":"+seconds;
 }
 else
 {
ans="0"+hours+":"+minutes+":0"+seconds;
 }
 }
 else
 {
 if(seconds>9)
 {
ans="0"+hours+":0"+minutes+":"+seconds;
 }
 else
 {
ans="0"+hours+":0"+minutes+":0"+seconds;
 }
 }
 }
 return ans;
}
}
```

Output :

```
Command Prompt

C:\Users\Admin\Documents>javac eg1.java
Note: eg1.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\Admin\Documents>java Practical1
Enter a Time:3405
Time is :00:56:45

C:\Users\Admin\Documents>
```

Conclusion :

Hence,from this practical we have learned m to convert time in seconds to hours, minutes and seconds and display the output in format HH:MM:SS .

Practical 2

Aim :Write a Java program to rearrange all the elements of a given array of integers so that all the odd numbers come before all the even numbers.

Theory :

Normally, an array is a collection of similar type of elements which has contiguous memory location.

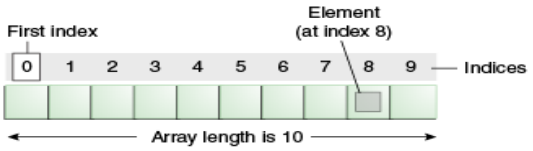
Java array is an object which contains elements of a similar data type. Additionally, The elements of an array are stored in a contiguous memory location. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in Java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index and so on.

Unlike C/C++, we can get the length of the array using the length member. In C/C++, we need to use the sizeof operator.

In Java, array is an object of a dynamically generated class. Java array inherits the Object class, and implements the Serializable as well as Cloneable interfaces. We can store primitive values or objects in an array in Java. Like C/C++, we can also create single dimentional or multidimentional arrays in Java.

Moreover, Java provides the feature of anonymous arrays which is not available in C/C++.



Advantages

- Code Optimization: It makes the code optimized, we can retrieve or sort the data efficiently.
- Random access: We can get any data located at an index position.

Disadvantages

- Size Limit: We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

Syntax to Declare an Array in Java

dataType[] arr; (or)
dataType[]arr; (or)
dataTypearr[];

Instantiation of an Array in Java

arrayRefVar=new datatype[size];

Program :

```
import java.io.DataInputStream;
import java.io.IOException;
class Practical2
{
public static void main(String args[])throws IOException
{
DataInputStream dis=new DataInputStream(System.in);
System.out.print("Enter the Length of the array:");
 int len=Integer.parseInt(dis.readLine());
 int arr[]=new int[len];
System.out.println("Enter the elements :");
for(int i=0;i<len;i++)
 {
arr[i]=Integer.parseInt(dis.readLine());
 }
 new Practical2().sort(arr);
System.out.println("Sorted array is :");
for(int i=0;i<len;i++)
 {
System.out.print(""+arr[i]+"\t");
 }
}
```

```
public void sort(int arr[])
{
int lb=0;
int ub=0;
while(ub<arr.length)
{
 if(arr[ub]%2!=0)
 {
 int temp=arr[lb];
arr[lb]=arr[ub];
arr[ub]=temp;
lb++;
 }
ub++;
}
}
}
```

Output :

```
C:\Users\Admin\Documents>javac eg1.java
Note: eg1.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\Admin\Documents>java Practical2
Enter the Length of the array:5
Enter the elements :
23
53
22
142
1
Sorted array is :
23      53      1       142     22
C:\Users\Admin\Documents>
```

Conclusion :

Hence, from this practical we have learned how to rearrange all the elements of a given array of integers so that all the odd numbers come before all the even numbers.

Practical 3

Aim :Write a Java program that accepts three integers from the user and return true if two or more of them (integers) have the same rightmost digit. The integers are non- negative

Theory :

Scanner class in Java is found in the java.util package. Java provides various ways to read input from the keyboard, the java.util.Scanner class is one of them.

The Java Scanner class breaks the input into tokens using a delimiter which is whitespace by default. It provides many methods to read and parse various primitive values.

The Java Scanner class is widely used to parse text for strings and primitive types using a regular expression. It is the simplest way to get input in Java. By the help of Scanner in Java, we can get input from the user in primitive types such as int, long, double, byte, float, short, etc.

The Java Scanner class extends Object class and implements Iterator and Closeable interfaces.

The Java Scanner class provides nextXXX() methods to return the type of value such as nextInt(), nextByte(), nextShort(), next(), nextLine(), nextDouble(), nextFloat(), nextBoolean(), etc. To get a single character from the scanner, you can call next().charAt(0) method which returns a single character.

Java Scanner Class Declaration

```
public final class Scanner
        extends Object
        implements Iterator<String>
```

How to get Java Scanner

To get the instance of Java Scanner which reads input from the user, we need to pass the input stream (System.in) in the constructor of Scanner class. For Example:

Scanner in = new Scanner(System.in);
To get the instance of Java Scanner which parses the strings, we need to pass the strings in the constructor of Scanner class. For Example:

Scanner in = new Scanner("Hello Java");

Program :

```
import java.io.DataInputStream;
import java.io.IOException;
class Practical3
{
public static void main(String args[])throws IOException
{
DataInputStream dis=new DataInputStream(System.in);
 int n1,n2,n3;
System.out.print("Enter the First Number :");
 n1=Integer.parseInt(dis.readLine());
System.out.print("Enter the Second Number :");
 n2=Integer.parseInt(dis.readLine());
System.out.print("Enter the Third Number :");
 n3=Integer.parseInt(dis.readLine());
 if((n1%10)==(n2%10) || (n1%10)==(n3%10) || (n2%10)==(n3%10))
  {
System.out.println("True");
  }
 else
  {
System.out.println("False");
  }
}
}
```

Output :

```
C:\Users\Admin\Documents>javac eg1.java
Note: eg1.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Users\Admin\Documents>java Practical3
Enter the First Number :12
Enter the Second Number :23
Enter the Third Number :43
True

C:\Users\Admin\Documents>
```

Conclusion :

Hence, from this practical we haveperformed a Java program that accepts three integers from the user and return true if two or more of them (integers) have the same rightmost digit. The integers are non- negative.

Practical 4

Aim: Create a Java class called Complex with the following details as member variables within it. (i) Real (ii) Imaginary. Develop a Java program to perform addition and subtraction of two complex numbers by using the method add() and subtract( ) respectively, by passing object as parameter and display result using method display(). Initialize the real and imaginary values of the complex number using parameterized constructor. Also demonstrate overloading constructors and methods.

Theory :

Objects in Java

If we consider the real-world, we can find many objects around us, cars, dogs, humans, etc. All these objects have a state and a behavior.

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging the tail, running.

If you compare the software object with a real-world object, they have very similar characteristics.

Software objects also have a state and a behavior. A software object's state is stored in fields and behavior is shown via methods.

So in software development, methods operate on the internal state of an object and the object-to-object communication is done via methods.

Classes in Java

A class is a blueprint from which individual objects are created.

A class can contain any of the following variable types.

Local variables − Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.

Instance variables − Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.

Class variables − Class variables are variables declared within a class, outside any method, with the static keyword.

A class can have any number of methods to access the value of various kinds of methods. In the above example, barking(), hungry() and sleeping() are methods.

Program :

```
import java.io.DataInputStream;
import java.io.IOException;
class Practical4
{
 public static void main(String args[])
 {
 Complex c1=new Complex(23,43);
 Complex c2=new Complex(332,412);

System.out.println("First Complex number: "+c1.display());
System.out.println("Second Complex number: "+c2.display());
 c1.add(c2);
System.out.println("Addition of Complex number:"+c1.display());
 c1.sub(c2);
System.out.println("Substraction of Complex number:"+c1.display());
 }
}
class Complex
{
 private int real;
 private int imaginary;
 public Complex(int r,inti)
 {
 real=r;
 imaginary=i;
 }
 public int getReal()
```

```
 {
 return real;
 }
 public int getImaginary()
 {
 return imaginary;
 }
 public String display()
 {
 return ""+real+"+i"+imaginary;
 }
 public void add(Complex c)
 {
this.real=this.real+c.getReal();
this.imaginary=this.imaginary+c.getImaginary();
 }
 public void sub(Complex c)
 {
this.real=this.real-c.getReal();
this.imaginary=this.imaginary-c.getImaginary();
 }
}
```

Output :

```
C:\Users\Admin\Documents>javac eg1.java

C:\Users\Admin\Documents>java Practical4
First Complex number: 23+i43
Second Complex number: 332+i412
Addition of Complex number:355+i455
Substraction of Complex number:23+i43

C:\Users\Admin\Documents>
```

Conclusion :

Hence, from this practical we have performed a Java class called Complex with the following details as member variables within it. (i) Real (ii) Imaginary. Develop a Java program to perform addition and subtraction of two complex numbers by using the method add() and subtract( ) respectively, by passing object as parameter and display result using method display(). Initialize the real and imaginary values of the complex number using parameterized constructor. Also demonstrate overloading constructors and methods.

Practical 5

Aim :Design an address with member variable num, city, state and country and appropriate constructor. Design a student class with constructor (student string USN, string name, string address) and employee class constructor (Employee string s, UPID, string name, string address).

Write a java program to create 'n' students object and print the student college and employee address respectively and demonstrate passing of object as a parameter to this constructor.

Theory:

Method in JAVA->

In JAVA, a method is like a function which is used to expose the behavior of an object.

Advantages of Methods:

1) Code Reusability

2) Code Optimization

3) New Keyword in Java

The new keyword is used to allocate memory at runtime and all objects gets memory in heap.

→By new keyword

→By new instance() method

Different ways to create object in Java

→By new instance() method

→By clone() method

→By deserialization method

→Factory method

*Anonymous object:

new collection(): anonymous object simply means nameless . An object which has no reference is known as anonymous objects.

Program:

```
import java.util.Scanner;

 class Address{

        int street_no;

        String city, state, country;


        Address(int street_no, String city, String state, String country){

        this.street_no = street_no;

        this.city = city;

        this.state = state;

        this.country = country;

        }

}


class Student{

        String usn, name;

        Address address;


        Student(String usn, String name, Address address){

        this.usn = usn;
```

```
        this.name = name;
        this.address = address;
        }
}

class College{
        String name;
        Address address;

        College(String name, Address address){
        this.name = name;
        this.address = address;
        }
}

class Employee{
        String id, name;
        Address address;

        Employee(String id, String name, Address address){
        this.id = id;
        this.name = name;
        this.address = address;
```

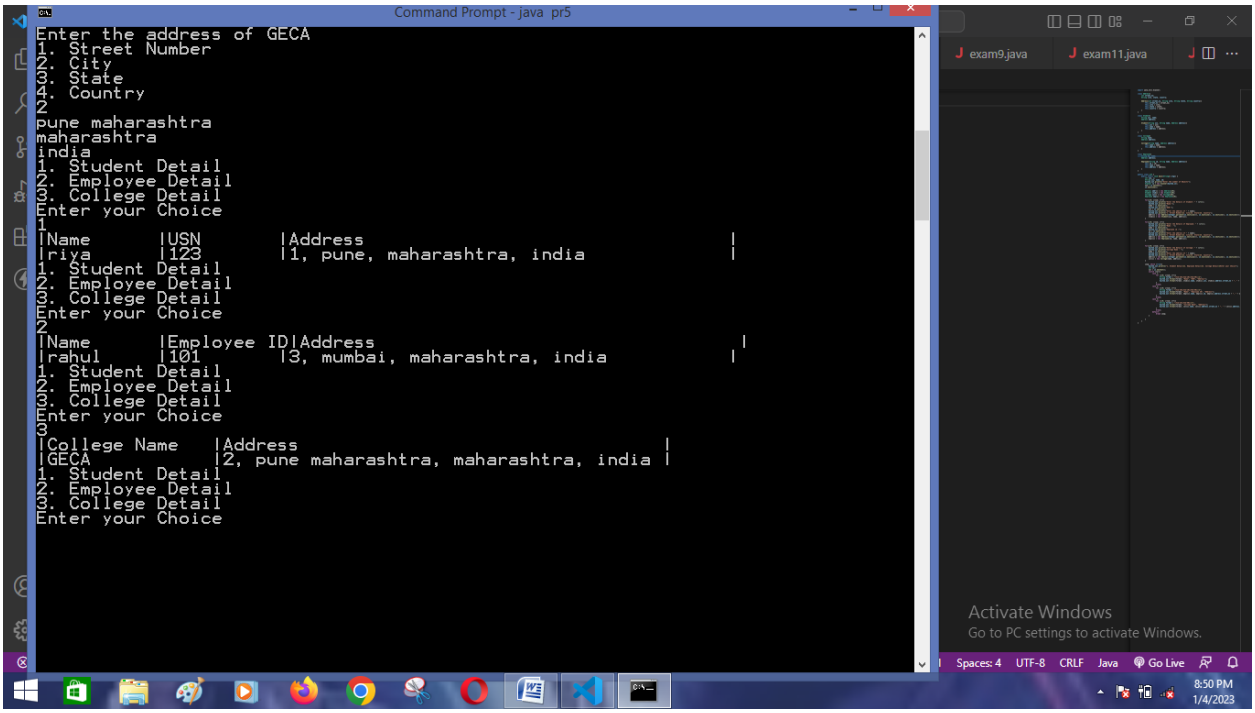
```
        }
}

public class pr5 {
        public static void main(String[] args) {
        int num, i;
          String usn, name, id;
        System.out.println("Enter the number of Objects");
        Scanner in = new Scanner(System.in);
        num = in.nextInt();
        in.nextLine();

        Address addr[] = new Address[10];
        Student stud[] = new Student[10];
        College col[] = new College[10];
        Employee empl[] = new Employee[10];

        for(i=0; i<num; i++){
        System.out.println("Enter the Details of Student: " + (i+1));
System.out.println("Name:");
        name = in.nextLine();
System.out.println("USN:");
        usn = in.nextLine();
```

```
System.out.println("Enter the address of " + name);

System.out.println("1. Street Number\n2. City\n3. State\n4. Country");

addr[i] = new Address(Integer.parseInt(in.nextLine()), in.nextLine(), in.nextLine(),
in.nextLine());

stud[i] = new Student(usn, name, addr[i]);

}


for(i=0; i<num; i++){

System.out.println("Enter the Details of Employee: " + (i+1));

System.out.println("Name : ");

name = in.nextLine();

System.out.println("Employee id :");

id = in.nextLine();

System.out.println("Enter the address of " + name);

System.out.println("1. Street Number\n2. City\n3. State\n4. Country");

addr[i] = new Address(Integer.parseInt(in.nextLine()), in.nextLine(), in.nextLine(),
in.nextLine());

empl[i] = new Employee(id, name, addr[i]);

}



for(i=0; i<num; i++){

System.out.println("Enter the Details of College: " + (i+1));

System.out.println("College Name : ");
```

```
name = in.nextLine();

System.out.println("Enter the address of " + name);

System.out.println("1. Street Number\n2. City\n3. State\n4. Country");

addr[i] = new Address(Integer.parseInt(in.nextLine()), in.nextLine(), in.nextLine(),
in.nextLine());

col[i] = new College(name, addr[i]);

}


Loop: while (true){

System.out.println("1. Student Detail\n2. Employee Detail\n3. College Detail\nEnter
your Choice");

int var;

var = in.nextInt();

switch (var){

case 1:

    for (i=0; i<num; i++){

String format = "|%1$-10s|%2$-10s|%3$-40s|\n";

System.out.format(format, "Name", "USN", "Address");

System.out.format(format, stud[i].name, stud[i].usn, stud[i].address.street_no + ", " +
stud[i].address.city + ", " + stud[i].address.state + ", " + stud[i].address.country);

}

break;

case 2:

for (i=0; i<num; i++){

   String format = "|%1$-10s|%2$-10s|%3$-40s|\n";
```

```
System.out.format(format, "Name", "Employee ID", "Address");

System.out.format(format, empl[i].name, empl[i].id, empl[i].address.street_no + ", " +
empl[i].address.city + ", " + empl[i].address.state + ", " + empl[i].address.country);

        }

        break;

        case 3:

        for (i=0; i<num; i++){

        String format = "|%1$-15s|%2$-40s|\n";

System.out.format(format, "College Name", "Address");

System.out.format(format, col[i].name, col[i].address.street_no + ", " + col[i].address.city + ", "
+ col[i].address.state + ", " + col[i].address.country);

        }

        break;

        default:

        break Loop;

        }



        }

        }

}
```

Output



Conclusion:

We have learned about ways to create an object in java using different methods.

Practical-6

Aim:-Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw Number Format Exception. If Num2 were Zero, the program would throwan Arithmetic Exception. Display the exception in a message dialog box..

Theory :- A class which is declared with abstract keyword is known as an abstract class in Java. It can have abstract and non abstract methods.

(Methods with the body)

Abstraction :-
It is a process of hiding the implement details and showing only functionality to the user and hide the internal details.

Abstract Class :-
It needs to be extended and it's method implements. It cannot be initialized.
1) An abstract class must be declared with abstract keyword.
2) It can have abstract and not abstract method.
3) It can be initialized.
4) It can have constructors and static method also.
5) It can have final method which will force the sub class not to change the body of the method. An abstract class can have a data member abstract method. Method body (Non-abstract method), constructor and even main () method.

Rules :-
1) if you extend abstract class that has an abstract method you must either produce the implementation of the method or make the class abstract.
2) If there is an abstract method in a class that class must be abstract.

PRGRAM:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="DivApplet" width=350 height=300>
</applet>
*/
public class DivApplet extends JApplet implements ActionListener{
JTextField number1,number2,result;
JButton divide;
public void init(){
try {
SwingUtilities.invokeAndWait(
new Runnable() {
public void run() {
makeGUI();
 }
 });
}
catch (Exception exc) {
System.out.println("Can't create because of " + exc);
 } }
 private void makeGUI(){
setLayout(new FlowLayout());
Label number1p = new Label("Number1: ",Label.RIGHT);
Label number2p = new Label("Number2: ",Label.RIGHT);
number1= new JTextField(20);
number2 = new JTextField(20);
result = new JTextField(20);
divide = new JButton("Divide");
add(number1p);
add(number1);
add(number2p);
add(number2);
add(result);
```

```
add(divide);
divide.addActionListener(this);
}
 public void actionPerformed(ActionEvent e){
 String snumber1,snumber2;
 snumber1 = number1.getText();
 snumber2 = number2.getText();
try{
 int number1 = Integer.parseInt(snumber1);
 int number2 = Integer.parseInt(snumber2);
 if(number2==0)
JOptionPane.showMessageDialog(null, "Division by zero not defined.");
else{
 double r = (double)number1/number2;
result.setText(((Double)r).toString());
}
 }
catch(NumberFormatException ne)
 {
JOptionPane.showMessageDialog(null,"Enter a number");
 }
 }
}
```

output:

Conclusion :-

By performing this practical we have learned how we can handle exception

Practical 7

Aim: Write a java program that implements multithreaded program with three threads.first thread genearates a random integer every one second and if the value is even , second thread computes the square of the number and prints.if the value is odd the third thread will print the value of cube of number

Theory:

Multithreading

Multithreading in jav**a** is a process of executing multiple threads simultaneously.

A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Advantages of Java Multithreading

1) It doesn't block the user because threads are independent and you can perform multiple operations at the same time.

2) You can perform many operations together, so it saves time.

3) Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

- Process-based Multitasking (Multiprocessing)
- Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

2) Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.

Life Cycle of a Thread

A thread goes through various stages in its life cycle. For example, a thread is born, started, runs, and then dies. The following diagram shows the complete life cycle of a thread.

Following are the stages of the life cycle −

- New − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a born thread.
- Runnable − After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- Waiting − Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

1	public void start() Starts the thread in a separate path of execution, then invokes the run() method on this Thread object.
2	public void run() If this Thread object was instantiated using a separate Runnable target, the run() method is invoked on that Runnable object.
3	public final void setName(String name) Changes the name of the Thread object. There is also a getName() method for retrieving the name.
4	public final void setPriority(int priority) Sets the priority of this Thread object. The possible values are between 1 and 10.
5	public final void setDaemon(boolean on) A parameter of true denotes this Thread as a daemon thread.
6	public final void join(long millisec) The current thread invokes this method on a second thread, causing the current thread to block until the second thread terminates or the specified number of milliseconds passes.
7	public void interrupt() Interrupts this thread, causing it to continue execution if it was blocked for any reason.
8	public final booleanisAlive() Returns true if the thread is alive, which is any time after the thread has been started but before it runs to completion.

Thread methods

Timed Waiting − A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

Terminated (Dead) − A runnable thread enters the terminated state when it completes its task or otherwise terminates.

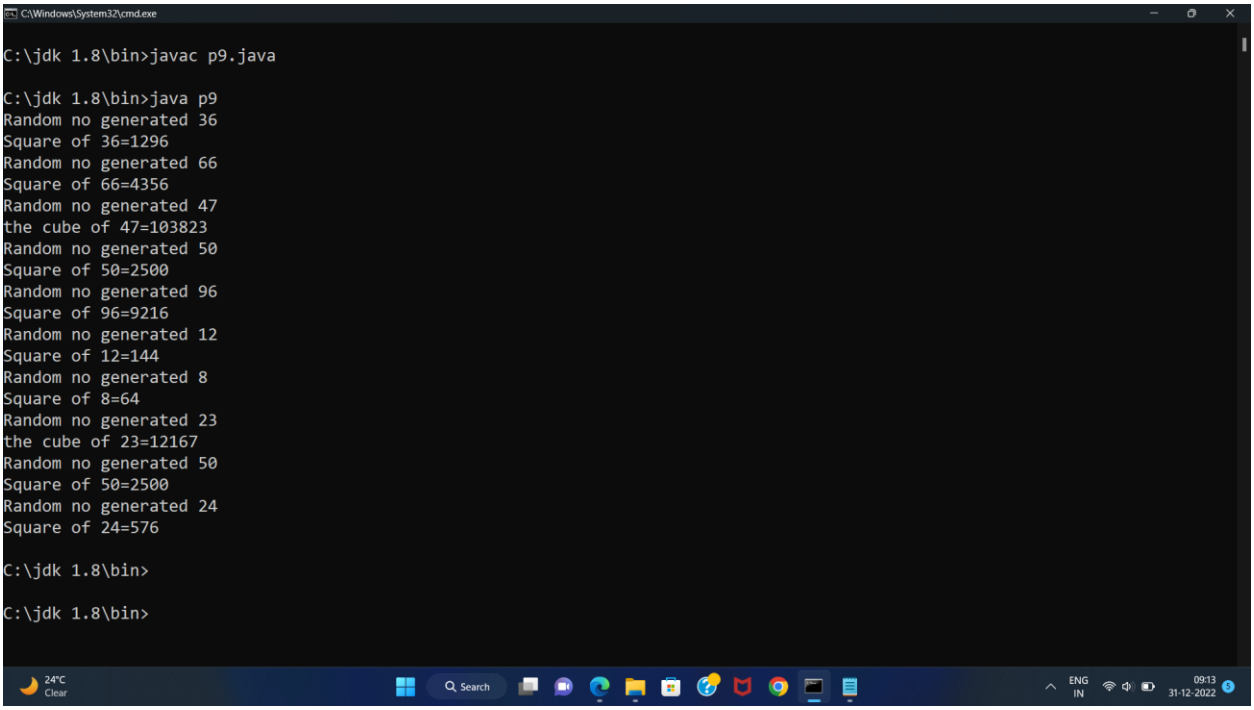
Program:

```
import java.util.Random;
class Square extends Thread {
   int x;
Square(int n) {
      x = n;
   }
   public void run() {
      int sqr = x * x;
System.out.println("Square of " + x + " = " + sqr);
   }
}
class Cube extends Thread {
   int x;
```

```
Cube(int n) {
        x = n;
    }
    public void run() {
        int cb = x * x * x;
System.out.println("Cube of " + x + " = " + cb);
    }
}
class Number extends Thread {
    public void run() {
        Random random = new Random();
        for (int i = 0; i< 10; i++) {
            int randomInteger = random.nextInt(100);
System.out.println("Random number generated: " + randomInteger);
            if (randomInteger % 2 == 0) {
                Square s = new Square(randomInteger);
s.start();
            } else {
                Cube c = new Cube(randomInteger);
c.start();
            }
            try {
Thread.sleep(1000);  // pause for 1 second
            } catch (InterruptedException e) {
```

```
System.out.println(e);
            }
        }
    }
}
public class P9 {
    public static void main(String[] args) {
        Number n = new Number();
n.start();
    }
}}
```

OUTPUT:



Conclusion : Thus we have implemented program that implements multithreaded program with three threads.first thread genearates a random integer every one second and

if the value is even , second thread computes the square of the number and prints.if the value is odd the third thread will print the value of cube of number

Practical No 10

Aim:-Write a Java Program to create an abstract class named shape that contains two integers a method named printArea().Create three classes named Rectangle,Triangle and circle such that each one of the classes extends the class shape.each one of the class contains only the method printArea() the area of the given shape

Theory:-Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

- Abstract class (0 to 100%)
- Interface (100%)

Abstract class in Java

A class which is declared as abstract is known as an abstract class. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example of abstract class

```
abstract class A{}
```

Abstract Method in Java

A method which is declared as abstract and does not have implementation is known as an abstract method.

Example of abstract method

```
abstract void printStatus();//no method body and abstract
```

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Itsimplementation is provided by the Honda class.

```
abstract class Bike{
  abstract void run();
}
class Honda4 extends Bike{
```

```
void run(){System.out.println("running safely");}

public static void main(String args[]){

 Bike obj = new Honda4();

obj.run();

}

}
```

Test it Now

running safely

Understanding the real scenario of Abstract class

In this example, Shape is the abstract class, and its implementation is provided by the Rectangle and Circle classes.

Mostly, we don't know about the implementation class (which is hidden to the end user), and an object of the implementation class is provided by the factory method.

A factory method is a method that returns the instance of the class. We will learn about the factory method later.

Why multiple inheritance is not possible in Java in case of class?

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields

Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Use of Inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Terms used in Inheritance

Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.

Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.

Super Class/Parent Class: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.

Reusability: As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

The extends keyword indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

In the terminology of Java, a class which is inherited is called a parent or superclass, and the new class is called child or subclass.

Java Inheritance Example

Inheritance in Java

As Programmer is the subclass and Employee is the superclass. The relationship between the two classes is Programmer IS-A Employee. It means that Programmer is a type of Employee.

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
 public static void main(String args[]){
   Programmer p=new Programmer();
System.out.println("Programmer salary is:"+p.salary);
System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

Test it Now

```
 Programmer salary is:40000.0
 Bonus of programmer is:10000
```

In the above example, Programmer object can access the field of own class as well as of Employee class i.e. code reusability.

Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.

Java Polymorphism

Polymorphism means "many forms", and it occurs when we have many classes that are related to each other by inheritance.

Like we specified in the previous chapter; Inheritance lets us inherit attributes and methods from another class. Polymorphism uses those methods to perform different tasks. This allows us to perform a single action in different ways.

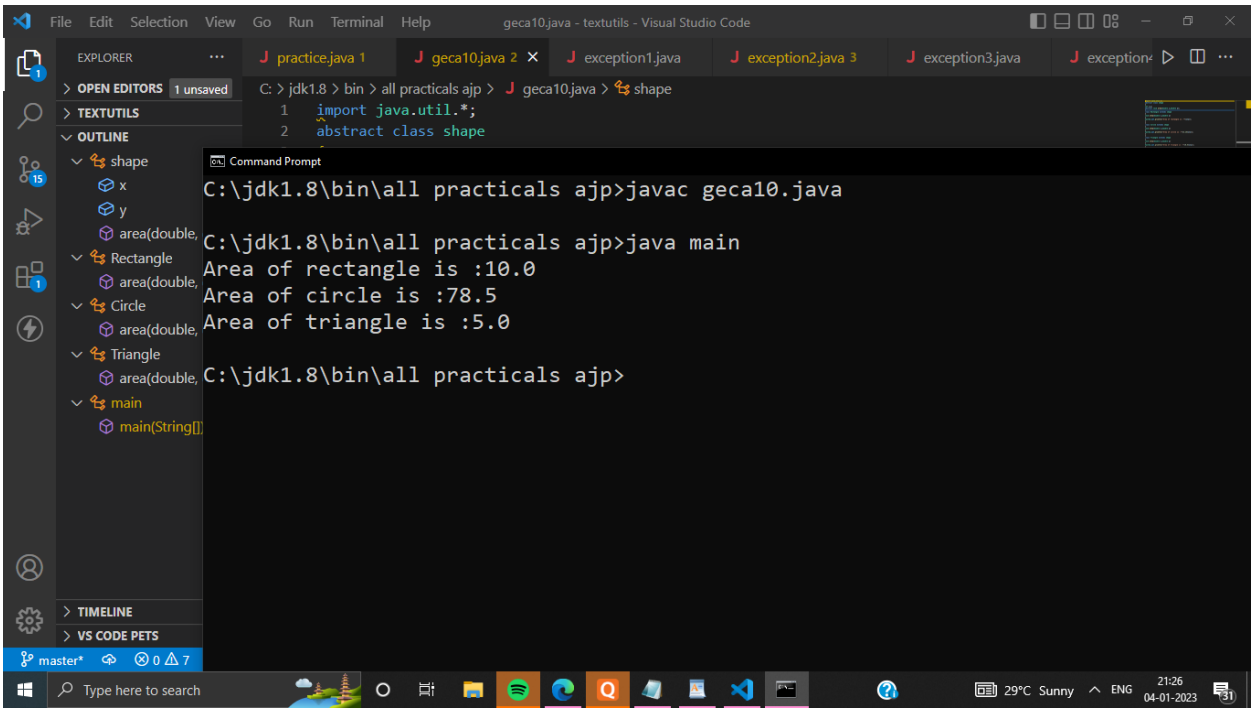
For example, think of a superclass called Animal that has a method called animalSound(). Subclasses of Animals could be Pigs, Cats, Dogs, Birds - And they also have their own implementation of an animal sound (the pig oinks, and the cat meows, etc.):

Program Code:

```
import java.util.*;

abstract class Shape {

   int x, y;

   abstract void area(double x, double y);

}

class Rectangle extends Shape {

   void area(double x, double y) {

System.out.println("Area of rectangle is: " + (x * y));

   }
```

```
}
class Circle extends Shape {
   void area(double x, double y) {
System.out.println("Area of circle is: " + (3.14 * x * x));
   }
}
class Triangle extends Shape {
   void area(double x, double y) {
System.out.println("Area of triangle is: " + (0.5 * x * y));
   }
}
public class AbstactDDemo {
   public static void main(String[] args) {
      Rectangle r = new Rectangle();
r.area(2, 5);
      Circle c = new Circle();
c.area(5, 5);
      Triangle t = new Triangle();
t.area(2, 5);
   }
}
```

Output:



Conclusion:

Thus we have created three classes Rectangle,Triangle and Circle we have extended class shape.also learnt about inheritance and their types and polymorphism.

Practical No 11

Aim: Write a java program for:1)Create a doubly linked list of elements.2)Delete a given element from above list.3)Display the contents of the list after deletions.

Theory:

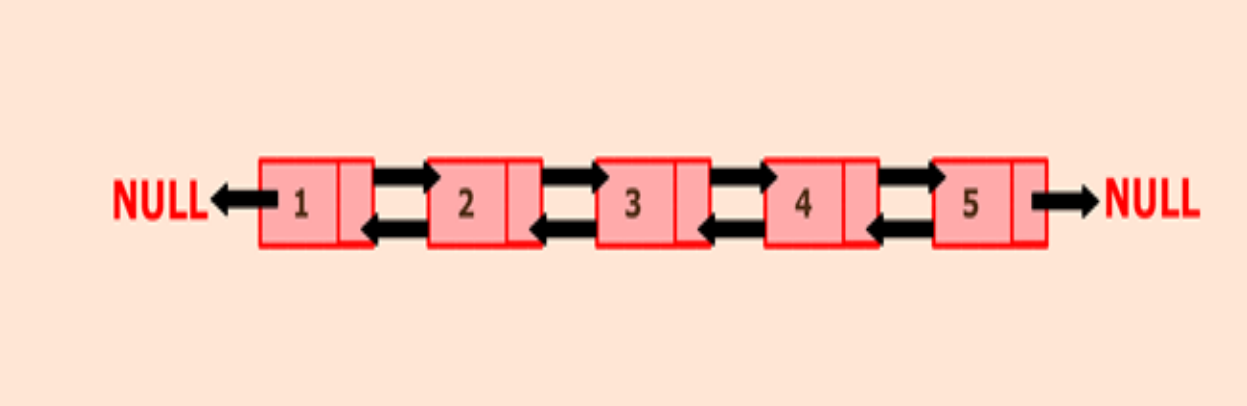
Doubly Linked List is a variation of the linked list. The linked list is a linear data structure which can be described as the collection of nodes. Nodes are connected through pointers. Each node contains two fields: data and pointer to the next field. The first node of the linked list is called the head, and the last node of the list is called the tail of the list.

One of the limitations of the singly linked list is that it can be traversed in only one direction that is forward. The doubly linked list has overcome this limitation by providing an additional pointer that points to the previous node. With the help of the previous pointer, the doubly linked list can be traversed in a backward direction thus making insertion and deletion operation easier to perform. So, a typical node in the doubly linked list consists of three fields:

Data represents the data value stored in the node.

Previous represents a pointer that points to the previous node.

Next represents a pointer that points to next node in the list.



Above picture represents a doubly linked list in which each node has two pointers to point to previous and next node respectively. Here, node 1 represents the head of the list. The previous pointer of the head node will always point to NULL. Next pointer of node

one will point to node 2. Node 5 represents the tail of the list whose previous pointer will point to node 4, and next will point to **NULL**..

Algorithm

- Define a Node class that represents a node in the linked list. It should have 3 properties i.e. previous node, data, and the next node
- Define another class to create a Doubly Linked List with two nodes i.e head and tail. Initially, these values will be null.
- Create a function for adding nodes in the linked list,
- It will first check whether the head is null and then insert the node as the head.
- Both head and tail will then point to the new node.
- If the tail is not null, the new node will be inserted at the list end in such a way that the pointer of the new node will point to the tail.
- Thus, the new node will become a new tail.

Basic Operations of Doubly Linked List:

- Insertion: Adding an element at the beginning of the linked list
- Deletion: Deleting an element at the beginning of the linked list
- Insert After: Adding an element after an item of linked list
- Insert Last: Adding an element to the end of the linked list
- Delete Last: Deleting an element at the end of linked list
- Delete: Deleting an element from the linked list using a key.
- Display forward: Displaying complete list in forward manner
- Display backward: Displaying complete list in backward manner

Advantages of DLL over the singly linked list:

- A DLL can be traversed in both forward and backward directions.
- The delete operation in DLL is more efficient if a pointer to the node to be deleted is given.
- We can quickly insert a new node before a given node.
- In a singly linked list, to delete a node, a pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using the previous pointer.

Program:

```
import java.util.*;
public class oops11 {
public static void main(String[] args)
{
 int i,ch,element,position;
 LinkedList dblList = new LinkedList();
System.out.println("1.Insert element at begining");
System.out.println("2.Insert element at end");
System.out.println("3.Insert element at position");
System.out.println("4.Delete a given element");
System.out.println("5.Display elements in the list");
System.out.println("6.Exit");
 Scanner sc=new Scanner(System.in);
 do { System.out.print("Choose your choice(1 - 6) :");
ch=sc.nextInt();
 switch(ch){
 case 1: // To read element form the user
System.out.print("Enter an element to insert at begining : ");
element=sc.nextInt(); // to add element to doubly linked list at
beginingdblList.addFirst(element);
System.out.println("Successfully Inserted");
Break;
 case 2: // To read element form the user
```

```
System.out.print("Enter an element to insert at end : ");
 element=sc.nextInt(); // to add element to doubly linked list at end
dblList.addLast(element);
System.out.println("Successfully Inserted");
 break;
case 3: // To read position form the user
System.out.print("Enter position to insert element : ");
position=sc.nextInt(); // checks if the position is lessthan or equal to list size.
if(position<=dblList.size()) { // To read element
System.out.print("Enter element : ");
element=sc.nextInt(); // to add element to doubly linked list at given position
dblList.add(position,element);
System.out.println("Successfully Inserted");
} else {
System.out.println("Enter the size between 0 to"+dblList.size());
 } break;
 case 4: // To read element form the user to remove
System.out.print("Enter element to remove :");
Integer ele_rm;
ele_rm=sc.nextInt();
if (dblList.contains(ele_rm))
{ dblList.remove(ele_rm);
System.out.println("Successfully Deleted");
Iterator itr=dblList.iterator();
System.out.println("Elements after deleting :"+ele_rm);
while(itr.hasNext()) {
System.out.print(itr.next()+"<->");
 }
System.out.println("NULL");
 } else {
System.out.println("Element not found");
 } break;
 case 5: // To Display elements in the list Iterator
itr=dblList.iterator();
System.out.println("Elements in the list :");
 while(itr.hasNext())
{ System.out.print(itr.next()+"<->");
```

```
}
System.out.println("NULL");
Break;
 case 6: System.out.println("Program terminated");
break ;
Default:
System.out.println ("Invalid choice");
 }
 }
 while (ch!=6); }
 }
```

Output:

```
C:\jdk 1.8\bin>javac oops11.java

C:\jdk 1.8\bin>java oops11
1.Insert element at begining
2.Insert element at end
3.Insert element at position
4.Delete a given element
5.Display elements in the list
6.Exit
Choose your choice(1 - 6) :1
Enter an element to insert at begining : 1
Successfully Inserted
Choose your choice(1 - 6) :2
Enter an element to insert at end : 3
Successfully Inserted
Choose your choice(1 - 6) :3
Enter position to insert element : 2
Enter element : 1
Successfully Inserted
Choose your choice(1 - 6) :5
Elements in the list :
1<->3<->1<->NULL
Choose your choice(1 - 6) :4
Enter element to remove :1
Successfully Deleted
Elements after deleting :1
3<->1<->NULL
Choose your choice(1 - 6) :
```

Conclusion:

In this practical we create doubly linked list of elements and delete given elements from the above list and display them.

Practical No 12

Aim: Write a java program that implements quicksort for sorting list of names in ascending order.

Theory:

QuickSort is a Divide and Conquer algorithm. It picks an element as a pivot and partitions the given array around the picked pivot. There are many different versions of quickSort that pick pivot in different ways.

- Always pick the first element as a pivot.
- Always pick the last element as a pivot (implemented below)
- Pick a random element as a pivot.
- Pick median as the pivot.

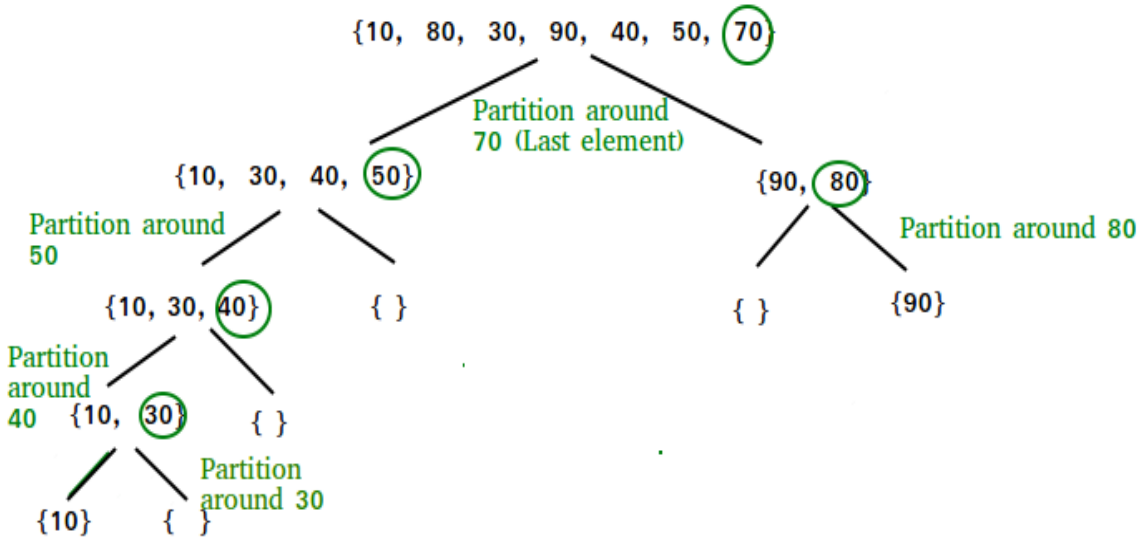
The key process in quickSort is a partition(). The target of partitions is, given an array and an element x of an array as the pivot, put x at its correct position in a sorted array and put all smaller elements (smaller than x) before x, and put all greater elements (greater than x) after x. All this should be done in linear time.

Steps to implement Quick sort:

1) Choose an element, called pivot, from the list. Generally pivot can be the last index element.

2) Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it . After this partitioning, the pivot is in its final position. This is called the partition operation.

3) Recursively apply the above steps to the sub-list of elements with smaller values and separately the sub-list of elements with greater values.



It is an algorithm of Divide & Conquer type.

Divide: Rearrange the elements and split arrays into two sub-arrays and an element in between search that each element in left sub array is less than or equal to the average element and each element in the right sub- array is larger than the middle element.

Conquer: Recursively, sort two sub arrays.

Combine: Combine the already sorted array.

Quick Sort Performance Analysis

Advantages

- As the name suggests !!! Is quick as a horse !!
- Reliable algorithm and used a lot in the industry

Disadvantages

- Needs additional spaces for temporary arrays, thus space complexity is O(log n)
- Very difficult to understand
- If the first element is chosen as a pivot by some idiot then it causes worst-case complexity of O(n^2)

Program:

```
import java.util.Arrays;
class Quicksort
{
(int array[], int low, int high)
{ // choose the rightmost element as pivot
int pivot = array[high]; // pointer for greater element
int i = (low - 1); // traverse through all elements
for (int j = low; j < high; j++) {
if (array[j] <= pivot) {
ii++;
// swapping element at i with element at j
int temp = array[i];
array[i] = array[j];
array[j] = temp;
 }
 int temp = array[i + 1];
array[i + 1] = array[high];
array[high] = temp; // return the position from where partition is done
return (i + 1);
 }
static void quickSort(int array[], int low, int high) {
if (low < high) {
// elements smaller than pivot are on the left
int pi = partition(array, low, high);
// recursive call on the left of pivot
quickSort(array, low, pi - 1);
// recursive call on the right of pivot
```

```
quickSort(array, pi + 1, high);
}
}
}
{
public static void main(String args[]) {
int[] data = { 8, 7, 2, 1, 0, 9, 6 };
System.out.println("Unsorted Array");
System.out.println(Arrays.toString(data));
int size = data.length;
Quicksort.quickSort(data, 0, size - 1);
System.out.println("Sorted Array in Ascending Order ");
System.out.println(Arrays.toString(data));
}
}
```

Output:

```
C:\jdk 1.8\bin>javac QuickMain.java

C:\jdk 1.8\bin>java QuickMain
Unsorted Array
[8, 7, 2, 1, 0, 9, 6]
Sorted Array in Ascending Order
[0, 1, 2, 6, 7, 8, 9]

C:\jdk 1.8\bin>_
```

Conclusion:

In this practical we create java program that implements quicksort for sorting list of names in ascending order.

.